WEEKLY MINI PROJECT 03 · TEAM 4

Who Doesn't Know?

Work Package · Leader / Integration

Leader | Integration | Docs & Demo

PROJECT	TEAM LEADER	OWNER
Who Doesn't Know?	**Taima Watfeh**	**Taima Watfeh**

ROLE	AREA
Team leader	**Foundation · integration · docs · demo**

> *Day 0 you unblock everyone. Day 4 you bring it all together.*

1 Your Work at a Glance

#	Task	Main file(s)
T1	Create the repository and both apps	`backend/`, `frontend/`
T2	Confirm feature schemas and follow the class ID pattern	`backend/src/games/schemas/*`, `gameplay/schemas/*`, `tips/schemas/*`
T3	Write shared frontend files	`api/http.ts`, `types.ts`
T4	Create empty files for every member	all folders
T5	Daily check-in and PR reviews	GitHub
T6	Write the README	`README.md`
T7	Integration day	`docs/acceptance-checklist.md`
T8	Presentation and demo	`docs/presentation/`

Use the class pattern: copy the structure you already built in `projects` and `workspaces`. Each feature gets its own module, controller, service, DTOs and schema. Keep the controller thin; put rules in the service; return clear NestJS exceptions.

2 Your Tasks, Step by Step

T1 Create the repository and both apps (Day 0)

`backend/` · `frontend/` · `.env.example`

1. `nest new backend`; install `@nestjs/mongoose mongoose @nestjs/swagger class-validator class-transformer`.
2. `npm create vite@latest frontend -- --template react-ts`; install `react-router-dom`.
3. Connect Mongo with `MongooseModule.forRoot(process.env.MONGODB_URI)`.
4. Write `.env.example`: `MONGODB_URI, PORT, INTERNAL_TOOL_TOKEN, VITE_API_URL, VITE_USE_MOCKS=true`.
5. Create branches `main` and `develop`; protect `main`.

✔ Done when: every member cloned the repo and both apps start.

T2 Confirm feature schemas and follow the class ID pattern (Day 0)

`backend/src/games/schemas/*` · `backend/src/gameplay/schemas/*` · `backend/src/tips/schemas/*`

1. Keep schemas beside the feature that owns them: Game and Player under `games/schemas`; Question, RoundDecision and SuspectVote under `gameplay/schemas`; SecretTip under `tips/schemas`.
2. Use the class style: `@Schema()`, `@Prop()`, `SchemaFactory.createForClass()`, and `Document` types.
3. Do not add CounterService. When creating a numeric id, copy the class pattern: `const last = await model.findOne().sort({ id: -1 }).exec(); const id = (last?.id ?? 0) + 1;`.
4. Add the indexes listed in the table (unique ones included).
5. Each feature module registers its own models with `MongooseModule.forFeature()`, exactly like ProjectsModule and WorkspacesModule.
6. Keep ID generation inside the owning service, matching `ProjectsService` and `WorkspacesService`; no shared counter file.

✔ Done when: both backend developers can inject any model without editing your files.

T3 Write the shared frontend files (Day 0)

`src/api/http.ts` · `src/types.ts`

1. Copy the two shared code blocks from section 3 of this document exactly.
2. These files are frozen: only you change them, and you announce every change.

✔ Done when: Yazan, Nawar and Alaa can import `request`, `USE_MOCKS` and all types.

T4 Create empty files for every member (Day 0)

`backend/src/games` · `backend/src/gameplay` · `backend/src/tips` · `frontend/src/pages/*` · `frontend/src/api/*.api.ts` · `frontend/src/mocks/*.mock.ts`

1. Create empty NestJS modules: `GamesModule`, `GameplayModule`, `TipsModule`, imported in `AppModule`.
2. Create empty screen components (returning the screen name) and empty api / mock files with the names in the overview.
3. Add a dev route `/dev` that lists `YazanPreview`, `NawarPreview`, `AlaaPreview` (empty components each member fills in).

✔ Done when: nobody ever needs to create a file another person also touches.

T5 Daily check-in and PR reviews (Days 1–5)

`GitHub`

1. 10-minute check-in each day: done / next / blocked.
2. Review each PR: only own files touched, no secret leaks in the public state, no game logic in the frontend.

✔ Done when: every PR into `develop` reviewed within the same day.

T6 Write the README

`README.md`

1. Install, environment variables and how to run both apps.
2. **Relationship decision**: separate collections linked by `gameId`, and why that keeps secrets out of the public state.
3. **Index decision**: `games.roomCode`, `players.tempId`, `players{gameId, normalizedName}`, unique decision / vote indexes, and why.
4. The temporary player id limitation: not authentication, not secure.

✔ Done when: a new person can install and run the project from the README alone.

T7 Integration day (Day 4)

`docs/acceptance-checklist.md`

1. Merge all branches into `develop`; make sure Mikdad's `GameplayReadService` replaced Hamwi's stub.
2. Set `VITE_USE_MOCKS=false` and run Flows A–E with 4+ browsers.
3. Run all 9 demo acceptance tests (section 18 of the spec), including a backend restart mid-game.
4. Open one ClickUp task per bug, assigned to the file owner.

✔ Done when: every CORE item and every acceptance test is ticked.

T8 Presentation and demo

`docs/presentation/`

1. Slides: game, walkthrough, data design decisions, secret hiding, why the temporary id is not authentication.
2. Speaking parts: Taima intro + data design · Hamwi ids & guard · Mikdad rules & ties · Yazan frontend architecture · Nawar game & voting demo · Alaa decision & result demo.
3. Prepare 4+ separate browsers / devices and rehearse once.

✔ Done when: full rehearsal done with every member speaking.

3 Shared Frontend Files You Write on Day 0

```
// frontend/src/api/http.ts  (Day-0 foundation, by Taima. Nobody edits it)
export const USE_MOCKS = import.meta.env.VITE_USE_MOCKS === 'true';
const BASE = import.meta.env.VITE_API_URL;

export async function request<T>(method: 'GET' | 'POST', path: string, body?: unknown): Promise<T> {
  const res = await fetch(BASE + path, { method, headers: { 'Content-Type': 'application/json' },
                                         body: body ? JSON.stringify(body) : undefined });
  const data = await res.json().catch(() => ({}));
  if (!res.ok) throw new Error(Array.isArray(data.message) ? data.message.join(', ') : data.message ??
'Request failed');
  return data as T;
}
export const mockDelay = <T,>(value: T, ms = 300) => new Promise<T>(r => setTimeout(() => r(value),
ms));
export const mockError = (message: string, ms = 300) =>
  new Promise<never>((_, rej) => setTimeout(() => rej(new Error(message)), ms));
```

```
// frontend/src/types.ts  (Day-0 foundation)
export interface Session { roomCode: string; playerId: string; playerRef: number; }
export interface ScreenProps {
  state: PublicGameState;      // from Yazan's polling hook
  me: PrivateRoleView;         // from Yazan's polling hook
  session: Session;            // send session to YOUR OWN api functions
  refresh: () => void;         // call after your action succeeds, to update immediately
}
```

```
// frontend/src/api/types.ts  (frozen contract, written in the Day-0 foundation)
export type GameStatus = 'lobby' | 'questioning' | 'round_decision' | 'voting' | 'finished';

export interface PublicPlayer { ref: number; displayName: string; isHost: boolean; }

export interface LogEntry { id: number; round: number; askerRef: number; targetRef: number;
  question: string; answer: string | null; }

export interface PublicGameState {
  gameId: number; roomCode: string; status: GameStatus; roundNumber: number; hostRef: number;
  players: PublicPlayer[]; minPlayers: number; playersNeeded: number;      // lobby
  turnOrder: number[]; currentPlayerRef: number | null; nextPlayerRef: number | null;
  turnsCompletedInRound: number; turnsRemainingInRound: number;
  pendingQuestion: { id: number; askerRef: number; targetRef: number; text: string } | null;
  log: LogEntry[];
  decision: { submittedCount: number; totalPlayers: number;
    lastOutcome: { round: number; continueCount: number; voteCount: number;
                   result: 'continue' | 'vote' } | null };
  voting: { submittedCount: number; totalPlayers: number;
    lastTie: { tiedRefs: number[]; counts: { ref: number; votes: number }[] } | null };
  result: null | { winner: 'informed' | 'hidden'; hiddenPlayerRef: number; secretTip: string;
    accusedRef: number; voteCounts: { ref: number; votes: number }[] };  // only when finished
}

export interface PrivateRoleView {
  playerRef: number; displayName: string; isHost: boolean;
  role: 'waiting' | 'informed' | 'hidden';   // 'waiting' while in lobby
  secretTip: string | null;                  // null for hidden player and in lobby
  hasDecided: boolean; hasVoted: boolean;    // for the current open decision / voting phase
}

export interface ApiError { statusCode: number; message: string; error: string; }
```